

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №5

Выполнил:

Машковцева Марина

К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

20226 г.

Задача

Подключить и настроить RabbitMQ для асинхронного взаимодействия между микросервисами. Заменить синхронные HTTP-запросы между User Service и Social Service на обмен сообщениями через очередь.

Ход работы

1. Выбор инструмента

Для реализации асинхронного взаимодействия был выбран RabbitMQ по следующим причинам:

- Простота настройки и использования
- Нативная поддержка Node.js (библиотека amqplib)
- Подходит для задач маршрутизации сообщений между сервисами

2. Установка и запуск RabbitMQ

RabbitMQ был запущен в Docker-контейнере:

```
docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:4.0-management
```

- Порт 5672: для подключения приложений
- Порт 15672: веб-интерфейс управления (логин: guest, пароль: guest)

3. Установка зависимостей

В user-service и social-service установлена библиотека:

```
npm install amqplib @types/amqplib
```

4. Реализация продюсера в User Service

Создан файл user-service/src/rabbitmq/producer.ts, который:

- Устанавливает соединение с RabbitMQ
- Создаёт очередь user_events
- Отправляет событие USER_CREATED после успешной регистрации пользователя

```
channel.sendToQueue('user_events', Buffer.from(JSON.stringify({
  event: 'USER_CREATED',
  data: { id, username, email }
})), { persistent: true });
```

5. Реализация консюмера в Social Service

Создан файл social-service/src/rabbitmq/consumer.ts, который:

- Подключается к той же очереди user_events
- Слушает входящие сообщения
- При получении события USER_CREATED сохраняет пользователя в локальную базу данных

```
channel.consume('user_events', async (msg) => {
  const content = JSON.parse(msg.content.toString());
  if (content.event === 'USER_CREATED') {
    await userRepo.save(content.data);
  }
  channel.ack(msg);
});
```

6. Создание модели User в Social Service

Для хранения копии пользователей создана модель:

```
@Entity()
export class User {
  @PrimaryColumn()
  id: number;
  @Column()
```

```
username: string;  
  
@Column()  
  
email: string;  
  
}
```

7. Обновление контроллеров Social Service

Все контроллеры (commentController.ts, likeController.ts, savedController.ts, subscriptionController.ts) были изменены:

- Было: синхронный HTTP-запрос к User Service (axios.get)
- Стало: проверка существования пользователя в локальной базе данных Social Service

Было

```
await axios.get(`http://localhost:3001/internal/users/${userId}`);
```

Стало

```
const user = await userRepo.findOneBy({ id: userId });
```

8. Интеграция в сервисы

В user-service/src/index.ts добавлен вызов подключения к RabbitMQ:

```
await connectRabbitMQ();
```

В social-service/src/index.ts добавлен запуск консюмера:

```
await consumeUserEvents();
```

Вывод

В ходе выполнения лабораторной работы был успешно реализован асинхронный обмен сообщениями между микросервисами с использованием RabbitMQ.